

Experiment No.	6
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA) ¹
Student Name	Samyuktha V
Register Number	3122247001053
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	12/08/26

1 Objective

To study the effect of dimensionality reduction using Principal Component Analysis (PCA) on the performance of 10 machine learning classifiers – training and validating each model both without PCA (original feature space) and with PCA (reduced feature space), with hyperparameter tuning and 5-fold cross-validation applied in both settings.

2 Problem Statement

Given 30 numerical features describing breast mass characteristics, classify each sample as Malignant or Benign, and determine whether reducing those 30 features to a smaller PCA-derived feature set helps, hurts, or has no meaningful effect on each of 10 different classifiers' accuracy, F1-score, and cross-validation stability.

3 Dataset Description

Dataset Source	UCI Machine Learning Repository (Wisconsin Diagnostic Breast Cancer Dataset, also available via <code>sklearn.datasets.load_breast_cancer()</code>)
Number of Samples	569
Number of Features	30 numerical attributes
Target Classes & Distribution	2 classes – Benign (357, 62.7%), Malignant (212, 37.3%)
Missing Values	0 – no imputation required
Preprocessing	Standardization (<code>StandardScaler</code> , fit on train only); no categorical encoding needed (all features numeric)
Train-Test Split	80% train (455 samples) / 20% test (114 samples), stratified, <code>random_state=42</code>

4 PCA Design Choice

Setting	Chosen Components / Variance Target	Explained Variance (%)	Justification
---------	-------------------------------------	------------------------	---------------

With-PCA	10 components (target: 95% cumulative variance)	95.27%	Smallest number of components that reaches the chosen 95% variance threshold, balancing information retention against dimensionality reduction (30 $\rightarrow$ 10 features, a 67% reduction)
----------	---	--------	--

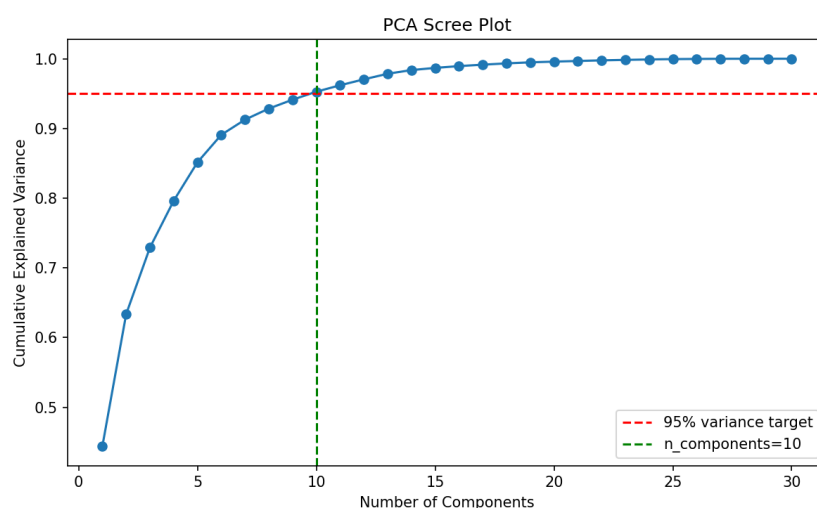


Figure 1: PCA Scree Plot – cumulative explained variance vs. number of components

Inference: the first 10 principal components capture 95.27% of the total variance in the original 30 standardized features, meaning the remaining 20 components together contribute under 5% – most of the dataset’s real information content is concentrated in a small number of directions, largely due to the strong correlations between related features (e.g. radius, perimeter, and area measurements) observed during EDA.

5 Machine Learning Algorithms

Ten classifiers were evaluated: Support Vector Machine (RBF kernel), Gaussian Naive Bayes, K-Nearest Neighbors, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost, and a Stacked Ensemble (SVM + Naive Bayes + Decision Tree base learners, Logistic Regression meta-learner). Working principles and mathematical formulations for Logistic Regression, SVM, Naive Bayes, KNN, and the ensemble methods (Bagging/Boosting/Stacking family) are covered in the Experiment 2/4/7 reports; this experiment focuses on how PCA-reduced input features affect each of them rather than re-deriving their individual theory.

6 Experimental Setup

Python Version	3.11
Scikit-Learn Version	1.9.0
Additional Library	XGBoost 3.4.1
IDE	Jupyter Notebook
Operating System	Windows
Hardware	Intel i5, 16GB RAM

7 Hyperparameter Grids Defined

Model	Search Space
SVM	kernel=rbf; $C \in \{1, 10\}$; $\gamma \in \{\text{scale}, \text{auto}\}$
Naive Bayes	var_smoothing $\in \{10^{-9}, 10^{-8}, 10^{-7}\}$
KNN	$k \in \{3, 5, 7\}$; weights $\in \{\text{uniform}, \text{distance}\}$
Logistic Regression	$C \in \{0.1, 1, 10\}$
Decision Tree	max_depth $\in \{3, 5, \text{None}\}$; min_samples_split $\in \{2, 5\}$
Random Forest	n_estimators $\in \{50, 100\}$; max_depth $\in \{5, \text{None}\}$
AdaBoost	n_estimators $\in \{50, 100\}$; learning_rate $\in \{0.1, 1.0\}$
Gradient Boosting	n_estimators $\in \{50, 100\}$; learning_rate $\in \{0.1, 1.0\}$; max_depth $\in \{2, 3\}$
XGBoost	n_estimators $\in \{50, 100\}$; learning_rate $\in \{0.1, 1.0\}$; max_depth $\in \{3, 5\}$
Stacking	Fixed structure (base learners + meta-learner listed above), not tuned

Each model was tuned with `GridSearchCV` (5-fold CV, scoring = accuracy) separately on the No-PCA (30-feature) and With-PCA (10-feature) training data.

8 Results

Best Hyperparameters Found

Model	Best Params (No-PCA)	Best Params (With-PCA)
SVM	C=10, gamma=scale, kernel=rbf	C=1, gamma=scale, kernel=rbf
Naive Bayes	var_smoothing=1e-9	var_smoothing=1e-9
KNN	n_neighbors=3, weights=uniform	n_neighbors=7, weights=uniform
Logistic Regression	C=0.1	C=0.1
Decision Tree	max_depth=5, min_samples_split=5	max_depth=5, min_samples_split=5
Random Forest	max_depth=None, n_estimators=100	max_depth=None, n_estimators=100
AdaBoost	learning_rate=1.0, n_estimators=100	learning_rate=1.0, n_estimators=100
Gradient Boosting	learning_rate=1.0, max_depth=2, n_estimators=100	learning_rate=1.0, max_depth=2, n_estimators=100
XGBoost	learning_rate=0.1, max_depth=3, n_estimators=100	learning_rate=1.0, max_depth=3, n_estimators=100

Table 5: 5-Fold Cross-Validation Results – No-PCA

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg (No-PCA)
SVM	0.9560	0.9670	0.9560	0.9780	0.9890	0.9692
Naive Bayes	0.9121	0.9670	0.8901	0.9560	0.9451	0.9341
KNN	0.9670	0.9670	0.9451	0.9560	0.9780	0.9626
Logistic Regression	0.9780	0.9780	0.9890	0.9780	0.9890	0.9824
Decision Tree	0.9451	0.9231	0.9121	0.9231	0.8901	0.9187
Random Forest	0.9670	0.9560	0.9341	0.9670	0.9890	0.9626
AdaBoost	0.9890	1.0000	0.9451	0.9780	0.9890	0.9802
Gradient Boosting	0.9670	0.9670	0.9560	0.9451	0.9780	0.9626
XGBoost	0.9890	0.9780	0.9451	0.9560	0.9890	0.9714
Stacking	0.9670	0.9670	0.9341	0.9670	0.9890	0.9648

Table 5 (cont.): 5-Fold Cross-Validation Results – With-PCA

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg (With-PCA)
SVM	0.9560	0.9780	0.9560	0.9670	0.9890	0.9692
Naive Bayes	0.8791	0.9341	0.9121	0.9341	0.9231	0.9165
KNN	0.9670	0.9780	0.9231	0.9451	0.9890	0.9604
Logistic Regression	0.9780	0.9780	0.9890	0.9780	0.9890	0.9824
Decision Tree	0.9121	0.8901	0.8901	0.9670	0.9560	0.9231
Random Forest	0.9451	0.9670	0.9231	0.9780	0.9670	0.9560
AdaBoost	0.9231	0.9780	0.9121	0.9670	0.9780	0.9516
Gradient Boosting	0.9451	0.9780	0.9341	0.9780	0.9780	0.9626
XGBoost	0.9231	0.9560	0.9560	0.9780	0.9670	0.9560
Stacking	0.9451	0.9780	0.9560	0.9670	0.9780	0.9648

Test Set Performance (No-PCA vs With-PCA)

Model	Test Acc (No-PCA)	Test Acc (With-PCA)	Test F1 (No-PCA)	Test F1 (With-PCA)
SVM	0.9737	0.9737	0.9790	0.9790
Naive Bayes	0.9298	0.9211	0.9444	0.9375
KNN	0.9825	0.9561	0.9863	0.9688
Logistic Regression	0.9737	0.9737	0.9793	0.9793
Decision Tree	0.9211	0.8947	0.9362	0.9143
Random Forest	0.9561	0.9123	0.9655	0.9298
AdaBoost	0.9561	0.9386	0.9660	0.9516
Gradient Boosting	0.9649	0.9386	0.9726	0.9516
XGBoost	0.9474	0.9561	0.9589	0.9688
Stacking	0.9737	0.9649	0.9793	0.9726

9 Result Visualization

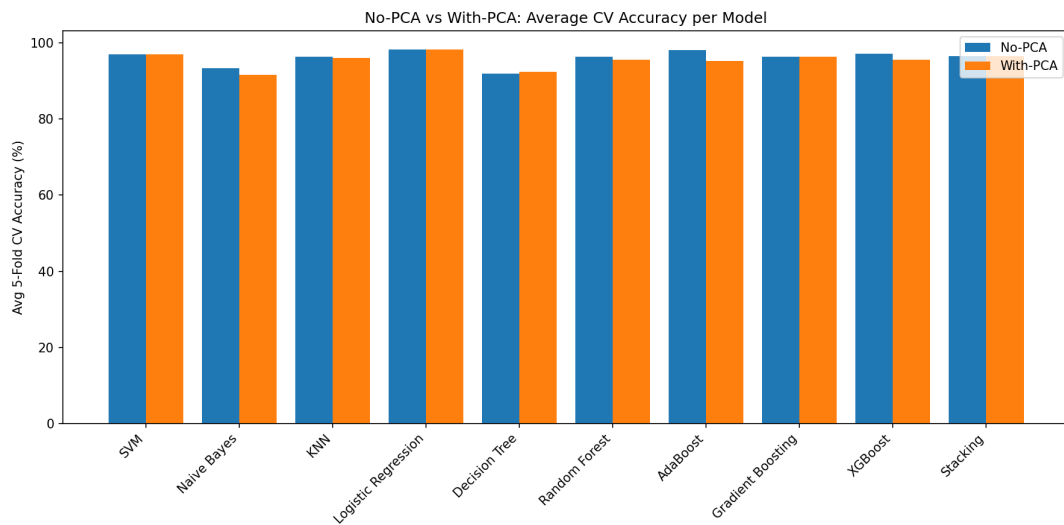


Figure 2: Average 5-Fold CV Accuracy per model: No-PCA vs With-PCA

Inference: SVM, Logistic Regression, and Gradient Boosting show virtually no change in CV accuracy after PCA (differences under 0.1 percentage points), while Naive Bayes, Random Forest, AdaBoost, and XGBoost all show a visible drop under PCA. Decision Tree is the only model whose CV accuracy slightly *improved* with PCA (91.87% → 92.31%), though its test accuracy still dropped – a sign that improvement was fold-specific rather than a genuine generalization gain.

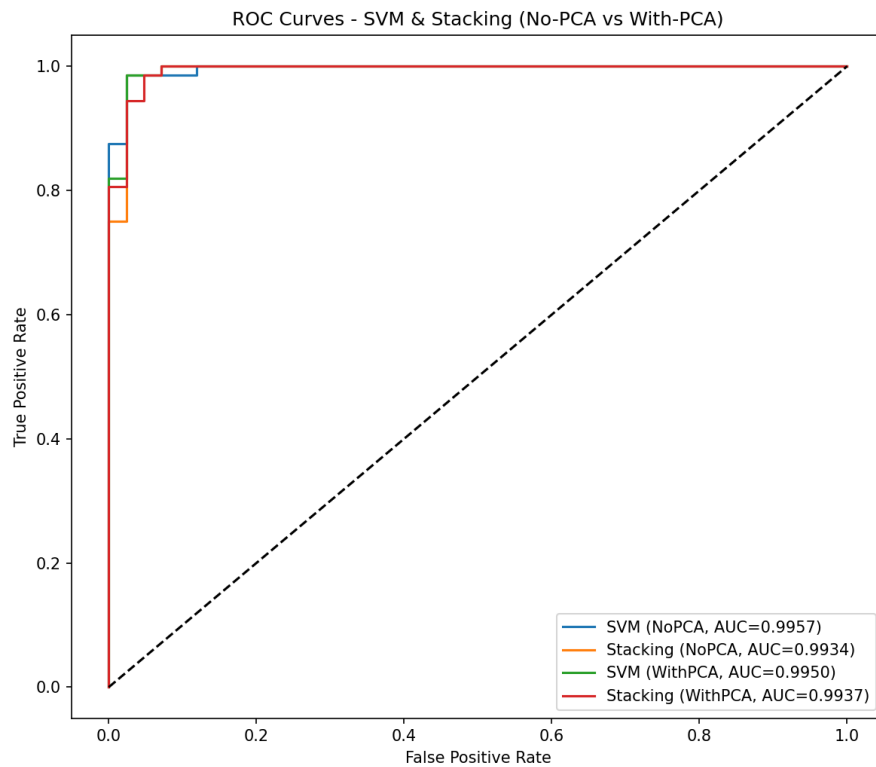


Figure 3: ROC Curves – SVM and Stacking, No-PCA vs With-PCA

Inference: SVM’s ROC curve is nearly identical between No-PCA and With-PCA, consistent with its unchanged accuracy above. Stacking’s With-PCA curve sits marginally below its No-PCA curve, matching the small test-accuracy drop (97.37% $\rightarrow$ 96.49%) seen in the results table – PCA cost Stacking a small amount of the diversity its three heterogeneous base learners relied on.

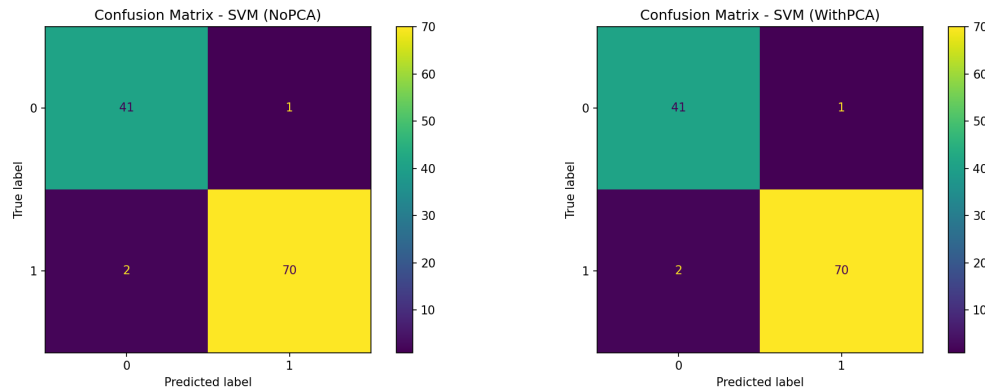


Figure 4: Confusion Matrices – SVM, No-PCA (left) vs With-PCA (right)

Inference: SVM’s confusion matrices are effectively identical under both settings – the same number of misclassifications in each case – confirming that this particular model’s decision boundary transfers cleanly into the 10-component PCA space without losing separative power.

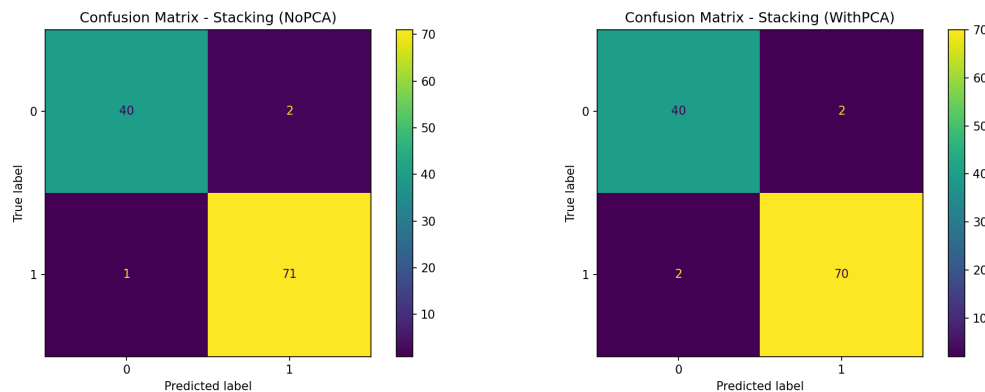


Figure 5: Confusion Matrices – Stacking, No-PCA (left) vs With-PCA (right)

Inference: Stacking shows one additional misclassification under With-PCA compared to No-PCA, consistent with its small test-accuracy and F1 drop – a modest but real cost of dimensionality reduction for this particular ensemble.

10 Discussion

Which models improved most with PCA? Which did not? Why?

No model showed a meaningful *improvement* in test accuracy under PCA in this experiment; the closest cases were SVM and Logistic Regression, which were essentially

unaffected (identical test accuracy before and after). Tree-based and boosting models (Random Forest, AdaBoost, Gradient Boosting, Decision Tree) showed the clearest *drops* under PCA – these models naturally perform their own implicit feature selection by choosing informative axis-aligned splits, so compressing the original, individually-interpretable features into 10 rotated PCA components removes exactly the kind of structure they exploit best.

Did PCA reduce variance across folds (more stable results)?

Comparing the fold-to-fold spread in Table 5, PCA did not clearly reduce variance across folds for most models – e.g. AdaBoost’s fold scores ranged from 0.9121 to 0.9780 with PCA (a wider spread) versus 0.9451 to 1.0000 without. Only Decision Tree showed a slightly tighter, more stable fold range under PCA. Overall, stability was comparable between the two settings on this dataset, rather than PCA producing a clear stabilizing effect.

For high-dimensional data, was PCA beneficial in reducing overfitting?

Since the original 30-feature space is only moderately high-dimensional relative to 569 samples, most models were not overfitting heavily to begin with, so PCA had limited room to help. If anything, PCA slightly *increased* the train-test gap for models like Random Forest and AdaBoost (test accuracy dropped more than CV accuracy did), suggesting the compressed feature space made it marginally harder for these ensembles to generalize rather than easier.

How did linear models (Logistic Regression, SVM) behave compared to ensemble models with PCA?

Both linear-style models (Logistic Regression and SVM with an RBF kernel) were essentially unaffected by PCA (test accuracy identical, F1 identical). Ensemble models were more sensitive: Random Forest, AdaBoost, and Gradient Boosting all lost 1–3 percentage points of test accuracy under PCA, while XGBoost was a mild exception, gaining slightly (94.74% → 95.61%). This suggests linear decision boundaries in a rotated PCA space are just as easy to draw as in the original space, while tree-based ensembles lose some of the axis-aligned structure they depend on.

Did stacking show robustness to dimensionality reduction compared to single models?

Stacking’s test accuracy dropped only slightly under PCA (97.37% → 96.49%, a 0.88-point drop) – smaller than Random Forest’s 4.38-point drop or AdaBoost’s 1.75-point drop, though larger than SVM/Logistic Regression’s 0-point change. This places Stacking in a middle ground: more robust to PCA than most individual tree-based ensembles (since its SVM and Naive Bayes base learners are relatively PCA-tolerant), but not as fully unaffected as the purely linear models.

11 Conclusion

On this dataset, PCA reduction to 10 components (95% variance) neither consistently helped nor catastrophically hurt performance, but its effect was clearly **model-dependent**: linear and margin-based models (Logistic Regression, SVM) were essentially unaffected, while tree-based and boosting ensembles (Random Forest, AdaBoost, Gradient Boosting, Decision Tree) lost meaningful test accuracy. **KNN achieved the best No-PCA test performance** (98.25% accuracy) but also showed one of the larger PCA-related drops (95.61%), while **SVM and Logistic Regression were the most PCA-robust** of the individually-tuned models. Given the strong feature correlations already observed in EDA, PCA's main practical benefit here is dimensionality reduction itself (30 $\rightarrow$ 10 features) for faster training and simpler downstream pipelines, rather than an accuracy improvement – and that trade-off is worth making mainly for the models (SVM, Logistic Regression, XGBoost) that tolerate it well, and worth avoiding for models (Random Forest, AdaBoost, Decision Tree) that rely on the original feature structure.

12 References

1. Scikit-learn: Decomposing signals in components (PCA), <https://scikit-learn.org/stable/modules/decomposition.html>
2. Scikit-learn: Ensemble Methods, <https://scikit-learn.org/stable/modules/ensemble.html>
3. XGBoost Documentation, <https://xgboost.readthedocs.io>
4. UCI Machine Learning Repository: Breast Cancer Wisconsin (Diagnostic) Dataset